ScreenPilot: an agent that learns to run legacy apps through the screen

Frontier BOG/26 · Challenge 1 · Design doc · Alejandro · Python + Playwright · Core principle: **think once, replay many**

1. PROBLEM, ASSUMPTIONS, SCOPE

Goal: given a natural-language task plus a noVNC desktop (URL and password), operate a GUI-only app the way a new employee would, then report a status and a summary within **5 minutes**. Grading runs on an **app we have never seen**, on a clean desktop.

Assumptions (we check each against the README and the staff):

- We only get pixels. There is no DOM, no accessibility tree and no API. The UI may be in Spanish.
- One evaluation app gets **several tasks**, so what we learn on early tasks pays off on later ones.
- Most tasks are business flows built from the same pieces: menu, form, search, table, pop-up, export.
- `/__reset` and `/__check` give us a reliable test oracle during development.

Hard cases: unexpected modals and pop-ups, slow redraws, long or paginated tables, file-save dialogs, date and number formats, vague tasks, irreversible actions (delete), getting stuck in loops, and the 15:30 incident (we expect the layout or behavior to change).

Out of scope, and why:

- Hand-written routes per app: the evaluation app is unseen.
- Fine-tuning a vision model: there is no time.
- Exploring the whole app up front: it would use most of the 5-minute budget.
- Third-party computer-use agents: the rules forbid them.

2. APPROACH: THREE EXECUTION TIERS

T1 · Primitives
scripted · 0 tokens
Reusable building blocks that work on any app: `click_text`, `fill_form`, `open_menu`, `dismiss_popup`, `find_row`, `scroll_until`, `export`, `wait_stable`.

T2 · Learned routes
0–1 LLM calls
A route is a recorded chain of primitives with slots for the task's values, anchored on text labels. The router matches the task to a route, fills the slots, and replays it.

T3 · Explorer
LLM step loop
Runs when no route matches. Each step: screenshot → OCR → numbered marks → the model picks the next primitive → act → verify. A cheap model by default; a stronger one if stuck.

How it learns:

- When a T3 run succeeds, the route learner turns its trace into a reusable route: literal values become slots, and each step is tied to a screen fingerprint and text anchors.
- Routes are stored per app fingerprint (window title plus the menu's OCR text).
- A failed replay falls back to T3 from the step where it broke, and the route is repaired.

Usage tracking: every run logs the route, success, steps, time and tokens. The most-used routes get extra checks and tighter steps; failing routes are demoted.

4. TIME BUDGET AND GUARDRAILS FOR EACH 5-MINUTE TASK

- **0–15 s:** connect, fingerprint the app, match a route. **Replay:** aim for under 90 s. **Explore:** at most 45 steps.
- Each action waits for the screen to settle (frame diff). Each step is checked by OCR (expected text, a new row, a success toast).
- Loop detection: if the same screen and action repeat 3 times, try a different strategy.
- Deletes and other irreversible actions run only when the task asks for them explicitly.
- **Hard stop at 4:30:** return `fallida` with a partial summary in the README's format.

3. KEY DECISIONS AND THE ALTERNATIVES WE REJECTED

Decision	Chosen	Rejected, and why
Screen access	Python Playwright opens the noVNC page, takes canvas screenshots, and sends mouse and keyboard events	A raw VNC (RFB) client: we don't know if the port is exposed, and the evaluation apps are reached "the same way as the test app"
Perception	OCR plus OpenCV box detection, with numbered marks drawn on the screenshot	A vision model guessing raw coordinates: less precise, more tokens, and a possible conflict with the rules
Route anchors	Text label plus relative position plus screen fingerprint	Absolute (x, y) coordinates: they break when the resolution or layout changes, which is likely the incident
Learning	Learn by doing: record, generalize, replay	A full exploration pass first: too slow for 5 minutes on an unseen app
Task → route	Embeddings shortlist the top 3 routes, then one cheap LLM call picks a route and fills its slots	An LLM call for every click: cost and latency
Models	OpenRouter: a cheap vision model by default, escalate after 2 failed steps; hard cap on cost per task	A strong model for every step: a US$45 budget can't cover it
API	FastAPI with an asyncio worker pool; `POST /tasks` queues the task and returns in under 100 ms	Doing the work inside the request: breaks the 2-second limit
Storage	SQLite on a persistent volume (tasks, routes, run logs)	Keeping it in memory: lost on every redeploy
Deploy	Docker with Chromium on Railway, with a health check	Serverless: time limits and cold starts

5. PLAN TO MEASURE (BEFORE WE BUILD)

- **Harness:** for each case, call `/__reset` → `POST /tasks` → poll → `/__check`. The cases are the 5 example tasks, about 15 paraphrases, and versions with new slot values. It runs on every deploy.
- **Generalization:** hold out 2 tasks from route learning, and run replays with values never seen before.
- **Robustness:** test with injected pop-ups, a different window size, and slow redraws (our rehearsal for the incident).

Metric	Target by 20:00
Task success (`/__check`): first try with T3 / replay with T2	≥ 60% / ≥ 90%
Median time: T3 / T2	< 180 s / < 90 s
Cost per task: T3 / T2	< US$0.40 / < US$0.05
Route hit rate on repeated tasks; `POST` latency	≥ 80%; p99 < 500 ms

Every failure gets a category (perception, planning, timing, pop-up, verification), and we fix the largest category first.

Open questions for the staff: are model-native computer-use APIs allowed? Can tasks run at the same time on one desktop? How is a file export checked?